NILE
UNIVERSITY
Nile University

NILE JOURNAL OF COMPUTING
Volume 1, Issue 1, 2026

Smart Market Delivery Platform: An AI-Assisted Market-to-Home Ingredient Procurement System for Nigerian Consumers

Hajarah Ayomide Ibrahim[1]*, Obichukwu Chiagoziem Awanye[1], Munachimso Collins Nzenwa[1], Abasiekeme Uduak Bassey[1] and Aminu Nuhu[1]

[1]Department of Computer Science, Faculty of Natural and Applied Sciences, Nile University of Nigeria, Abuja, Nigeria

*Corresponding Author: Hajarah Ayomide Ibrahim (e-mail: hajbae01@gmail.com)

ABSTRACT: This paper proposes a Smart Market Delivery Platform (SMDP), a mobile-first service that connects meal planning with human-assisted shopping in Nigerian markets. A user enters a dish such as egusi soup, selects the number of servings, confirms the ingredients and quantities already available at home, and chooses a nearby participating market. The platform creates an editable list of missing ingredients, estimates the basket cost from timestamped market-price observations, and assigns a worker stationed at the selected market to purchase the requested items. Customers receive item-level progress updates, can approve substitutions or material price changes, and receive an itemized receipt when the order is delivered. The minimum viable product (MVP) is proposed to integrate an AI model API for recipe interpretation and ingredient normalization, a structured market-price database updated by workers, and Paystack for payment collection. An AI model should not be treated as a source of verified live prices; estimates must be grounded in recent observations and clearly labelled when uncertain. The paper presents the user journey, system architecture, requirements, payment controls, operating model, implementation roadmap, and proposed evaluation plan. No deployment or experimental performance results are claimed.

KEYWORDS: ingredient delivery, Nigerian markets, recipe scaling, artificial intelligence, Paystack, price estimation, market workers, last-mile delivery

I. INTRODUCTION

Preparing a meal requires more than choosing a recipe. A household must determine the ingredients and quantities needed, identify what is already available, purchase the remaining items, and transport them home. In many Nigerian settings, this can involve visiting an open market and buying from several vendors. The process may be inconvenient for students, working households, caregivers, and customers who cannot easily travel to a market.

The proposed platform reframes market information as a service that begins with a planned meal and ends with the delivery of its ingredients. Rather than asking users to search for every item individually, the application translates a dish into an editable shopping list, subtracts pantry quantities, estimates the cost, and coordinates a market-based worker who purchases and delivers the missing items.

1.1 Problem Statement

Consumers must translate recipes into practical shopping lists while dealing with variable serving sizes, ingredients already at home, market availability, and changing prices. Conventional recipe tools do not complete the purchase, while ordinary delivery applications may require users to search for each ingredient separately. The proposed system combines meal-based planning, market-worker fulfilment, price transparency, payment, and delivery in one workflow.

1.2 Aim and Objectives

- Generate an editable ingredient list from a selected dish and serving count.
- Allow users to specify pantry ingredients and available quantities so only missing items are purchased.
- Let customers choose a nearby participating market and assign a worker stationed there.
- Estimate costs from dated price observations and record actual prices during shopping.
- Integrate Paystack payment verification, order tracking, substitutions, receipts, and delivery confirmation.
- Define a controlled MVP pilot and measures for usability, price accuracy, fulfilment, and trust.

The platform must also address trust. Users need to know whether a price is an estimate or a confirmed purchase price, whether an item has actually been bought, and whether any substitution or cost change requires approval. These details are essential because the service involves both online transactions and real-world shopping by a third party.

1.3 Research Questions

The design is guided by four questions: (1) How can a meal description be converted into a useful shopping list? (2) How should the system estimate a basket total when market prices are variable and sometimes unavailable digitally? (3) How can users observe and approve decisions made by a market worker? (4) Which architecture and operating controls are appropriate for a small Nigerian-market MVP?

II. RELATED WORKS

2.1 Recipe and Ingredient Recommendation

Recipe applications commonly present ingredient lists and cooking instructions, while recommendation systems may personalize meal suggestions to user preferences. The proposed service uses a related ingredient-planning capability but extends it into purchasing and delivery. Ingredient names, units, and quantities must be structured so that the shopping list can be edited and priced reliably.

2.2 On-Demand Shopping and Last-Mile Delivery

On-demand delivery services coordinate customers, fulfilment agents, merchants, and transport. The proposed market-worker model differs from a conventional supermarket catalogue because workers may need to purchase items from multiple stalls in an open market. As a result, availability checks, item substitutions, quantity confirmation, receipt capture, and transparent order states are core functions rather than optional enhancements.

2.3 Price Estimation and Data Freshness

A price estimate is only useful when its source and age are understood. Market prices can vary by location, vendor, unit, quality, season, and time of day. An AI model can interpret language or help normalize ingredient names, but it cannot be assumed to know the current price at a specific stall. The MVP therefore needs a structured record of observed prices and worker-confirmed purchases.

2.4 Proposed Improvement over Existing Workflows

The principal contribution is an integrated workflow: meal selection, serving-size adjustment, pantry subtraction, market choice, price estimation, payment, human-assisted shopping, customer-approved substitutions, and delivery. This end-to-end approach makes the shopping task easier to understand and provides a record of how the final cost was reached.

TABLE I: Proposed Platform Capabilities

Function	Conventional workflow limitation	Proposed SMDP response
Recipe planning	Users manually translate recipes into shopping lists.	AI-assisted, editable ingredient list with serving-size scaling.
Pantry awareness	Users may purchase ingredients they already own.	Subtract user-entered available quantities.
Market purchase	Customer travels and buys from multiple vendors.	Assigned worker shops at the selected market.
Price transparency	Estimate sources and final differences may be unclear.	Dated observations, item-level actual prices, and final receipt.
Progress visibility	Customer may not know whether items were found or purchased.	Timestamped item statuses and approval requests.

III. METHODOLOGY

3.1 Research Design

This paper follows a design-oriented systems methodology. The proposed system is specified through a user journey, functional requirements, a logical architecture, a data model, and an operational workflow. The design is intended to be evaluated later through a controlled pilot rather than through assumed or fabricated performance results.

The initial validation should involve prospective customers and market workers. Interviews can establish which dishes users most often prepare, how they describe quantities, what price information they trust, which markets are practical, and what service fee they might accept. A manual fulfilment trial should precede automation so the team can test the core service before investing in a full application.

3.2 System Actors

Actor	Responsibilities
Customer	Choose meal, select servings, confirm pantry quantities, choose market, pay, approve substitutions, track order, confirm delivery, report problems.
Market worker	Accept assigned order, shop for approved items, record actual quantities and prices, flag unavailable goods, request approval for substitutions.
Operations administrator	Onboard workers, configure market coverage, reassign tasks, manage disputes, refunds, catalogue, fees, and support.
Payment provider	Process checkout and issue transaction events that the backend verifies before fulfilment.

3.3 Functional Requirements

- **Recipe interpretation:** accept a dish name and return a structured list of ingredients, quantities, units, and preparation assumptions.
- **Serving adjustment:** scale quantities to the requested servings and allow users to edit culturally or personally variable ingredients.
- **Pantry subtraction:** record what is already available and calculate the remaining quantity to buy.
- **Market selection:** show participating markets serving the delivery address, not merely the geographically closest market.
- **Basket estimate:** display item estimates, data timestamps, uncertainty, delivery fee, service fee, and possible variation.
- **Worker assignment:** route paid orders to eligible workers assigned to the selected market and support reassignment.
- **Live updates:** track item statuses such as pending, found, unavailable, substitution requested, purchased, or removed.
- **Approval:** pause affected purchases when a substitution or material price change requires customer consent.
- **Payment and delivery:** verify Paystack transactions, record final receipts, track delivery, and provide a support pathway.

3.4 Non-Functional Requirements

The system should be mobile-first, easy to use on common smartphones, and tolerant of intermittent connectivity. It must protect customer addresses and phone numbers, restrict workers to assigned orders, retain an auditable history of payment and order-state changes, and avoid storing raw payment-card details. Backend validation must govern order totals, payment status, permissions, refunds, and state transitions.

3.5 Technology Stack

A practical MVP may use a responsive web application or progressive web app, a Python backend such as Django REST Framework or FastAPI, and MySQL . The team should choose one backend framework and one database. The AI provider should sit behind a dedicated recipe service, while Paystack handles checkout. A maps/geocoding provider can be added for location support, although a small pilot can begin with manually configured delivery zones.

IV. SYSTEM DESIGN

4.1 User Journey and Application Workflow

The application starts with the customer's intended meal rather than a blank product catalogue. After entering a dish such as egusi soup, the user selects the desired serving count. The system proposes a recipe-specific ingredient list, after which the customer confirms which items are already at home and the amount available. The basket then contains only the missing quantities.

The customer selects a participating market, reviews estimated prices and fees, and confirms the order. Following verified payment, a worker assigned to that market receives the task. The worker updates each item while shopping, records actual quantities and prices, and requests approval when a product is unavailable or a substitution changes the basket materially. Once shopping is complete, the order is delivered and an itemized receipt is provided.

TABLE II: Main User and System Data Flows

Flow	Actor	Process	System response
F1: Meal input	Customer	Enters dish and servings.	Returns a structured, editable ingredient list.
F2: Pantry review	Customer	Enters available ingredient quantities.	Calculates missing quantities and removes covered items.
F3: Market choice	Customer	Selects a participating market.	Shows estimate, price freshness, fees, and coverage.
F4: Checkout	Customer / Paystack	Completes checkout.	Verifies transaction server-side before fulfilment.
F5: Market shopping	Worker	Records availability, quantities, prices, and substitutions.	Updates the customer timeline and approval requests.
F6: Delivery	Worker / customer	Hands over the purchased ingredients.	Closes order and produces itemized receipt.

4.2 Functional Architecture

The system is divided into customer and worker interfaces, application services, recipe intelligence, price and order storage, and external integrations. The backend remains the authority for order totals, payment verification, worker permissions, and state changes. This avoids placing critical business logic in the mobile or web client.

4.3 Core Data Entities

The main records are User, Address, Market, Worker, Ingredient, IngredientAlias, Recipe, RecipeIngredient, PriceObservation, Order, OrderItem, WorkerAssignment, PaymentTransaction, OrderEvent, SubstitutionRequest, Delivery, and SupportCase. Price observations should record the ingredient, market, unit, observed price, timestamp, and source. Each order item should preserve requested quantity, purchased quantity, agreed unit price, actual line total, and final status.

4.4 Order State Model

State	Meaning
Draft	Customer is editing the meal, quantities, market, or basket.
Awaiting payment	Customer has accepted the estimate and checkout is in progress.
Paid / awaiting assignment	Payment is verified and a worker must be assigned.
Shopping	Worker is buying items and recording item-level events.
Needs approval	Substitution or material cost change awaits customer response.
Purchased / out for delivery	Actual purchases are recorded and delivery is underway.
Delivered / completed	Handover is confirmed and receipt is available.
Cancelled / refund pending	Exception workflow records reason and payment reconciliation.

V. SYSTEM ARCHITECTURE

5.1 Proposed Smart Market Delivery Platform Architecture

Figure 1 shows the logical architecture for the revised platform. The customer interface collects meal preferences, servings, pantry quantities, market choice, and checkout confirmation. The application API validates input and coordinates recipe interpretation, basket calculations, order creation, worker assignment, and delivery states. The recipe service uses an AI model API for structured ingredient suggestions, while the market-price store contains dated observations and actual purchase records.

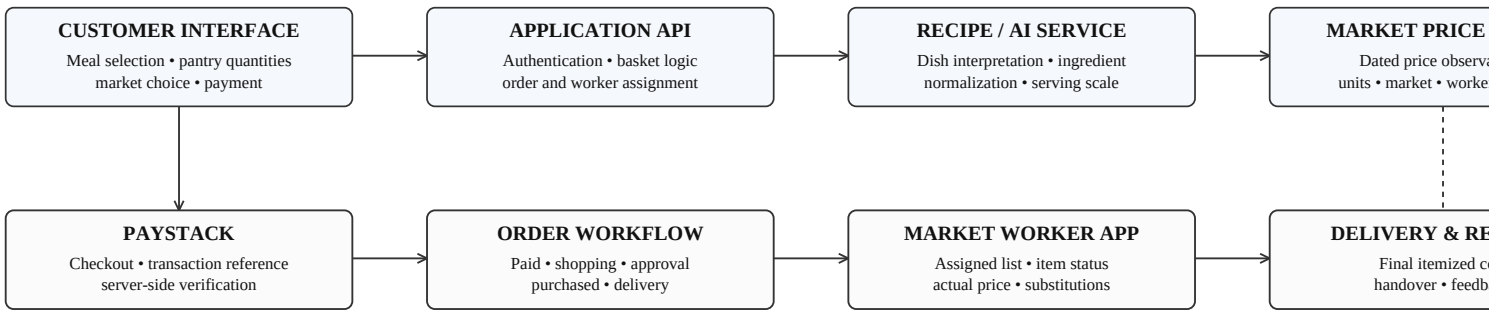


Figure 1. Proposed system architecture for the Smart Market Delivery Platform

5.2 Customer, Worker, and Operations Interfaces

The customer interface should show a clear progress timeline: list created, payment verified, worker assigned, shopping in progress, approval required where applicable, purchased, and out for delivery. The worker interface should prioritize a readable checklist, quantity and unit, item status, actual price entry, substitution requests, and receipt capture. An operations view is needed to handle unassigned orders, worker timeouts, customer disputes, and refunds.

5.3 Application Services

The application API coordinates authentication, recipe/list generation, pantry subtraction, basket estimation, market selection, worker assignment, payment verification, notification delivery, and order-state transitions. Critical functions must be idempotent where retries are possible. For example, receiving the same payment webhook more than once must not create duplicate shopping tasks.

5.4 Storage and Integration Layer

MySQL can store users, market coverage, ingredients, recipes, price observations, orders, payment references, and audit events. The Paystack API supports checkout and transaction verification. A maps service may assist with address entry and distance estimates; notification services can send status changes. The architecture should remain modular so the AI provider or notification vendor can be changed without replacing the order-management logic.

5.5 Technology Stack and Deployment

Layer	Proposed MVP technology	Design rationale
Client	Responsive web app / PWA	Quick to test with users and avoids early app-store overhead.
Backend	Django REST Framework or FastAPI	API validation, authorization, payment and workflow rules.
Database	MySQL	Relational consistency for orders, payments, and market observations.
AI service	External model API	Recipe interpretation and ingredient normalization only.
Payments	Paystack API	Checkout, reference verification, webhook-driven updates.
Location / alerts	Maps or defined zones; SMS/push	Market coverage and timely customer/worker updates.

VI. AI-ASSISTED INGREDIENT PLANNING AND PRICE DATA

6.1 Role of the AI Model API

For the MVP, the AI model should interpret a dish name and return a structured recipe suggestion. For example, “egusi soup for six people” can produce ingredient names, quantities, units, and recipe notes. The backend validates the output against a schema and maps ingredient names to a curated catalogue. The customer must be able to edit the list because recipes vary across households, regions, dietary preferences, and cooking methods.

The AI service should not independently control order totals, approve substitutions, or determine payment status. Those operations depend on validated application rules and actual market records. Recipe output should be saved with the order so that the shopping list used for fulfilment can be reviewed later if a dispute occurs.

6.2 Obtaining Reliable Price Data

An AI model API does not inherently provide verified, live prices for a particular stall in a particular market. If no reliable live price feed is available, asking a model for current prices may return plausible but incorrect figures. The platform should instead maintain its own market-price observations. Workers can record prices during purchases and periodically check common ingredients at participating markets.

Price source	Purpose	Display rule
Worker-confirmed purchase	Actual item price and quantity for the active order.	Show as confirmed for this order and include quantity/unit.
Recent market observation	Pre-order estimate for a common ingredient.	Show market and timestamp; label as estimate.
Historical observation	Fallback when a current observation is unavailable.	Show range or low-confidence label when stale.
AI-generated estimate	Optional fallback only when better data is missing.	Explicitly unverified; never presented as a live quote.

6.3 Basket Calculation

For each ingredient, the missing quantity is the required quantity minus the usable quantity the customer already has, with a minimum of zero. The estimated ingredient cost is the sum of each missing quantity multiplied by its estimated unit price after safe unit conversion. Delivery and service fees are shown separately. When prices are variable or data is sparse, the application should show an indicative range rather than implying false precision.

6.4 Example: Egusi Soup

A customer selects egusi soup and six servings. The system proposes a recipe-specific list, potentially including egusi, leafy vegetables, pepper, palm oil, stockfish or another chosen protein, seasoning, and other ingredients. The customer marks palm oil and seasoning as available and enters their quantities. The system calculates the missing amounts, asks for clarification where units are ambiguous, and estimates the basket from recent market observations. A worker then buys the approved items and records actual prices. If a selected vegetable is unavailable, the worker sends a substitute and its price for approval before proceeding.

6.5 Price Changes and Reconciliation

The platform must distinguish an estimated total from the final purchase total. The team should publish a clear payment policy, such as collecting a defined estimated amount and reconciling the final total through supported payment operations, or using a properly disclosed authorization and adjustment workflow. The worker must not silently buy a materially more expensive substitute or charge an unapproved amount.

VII. PAYMENT INTEGRATION AND SECURITY

7.1 Proposed Paystack Payment Flow

- The backend creates a pending order and calculates the estimate from server-side records.
- The backend initiates a Paystack transaction using the order reference, amount in the required currency unit, customer email, and appropriate metadata.
- The customer completes the supported checkout flow.
- The backend verifies the transaction and checks reference, amount, currency, and status before marking the order paid.
- Verified webhook events update transaction state; processing must be idempotent to prevent duplicate fulfilment.
- The system records the final itemized cost and reconciles any adjustment or refund under the published policy.

7.2 Security and Privacy

Secret API keys must remain on the server and must not be embedded in frontend code or source control. The application should use HTTPS, input validation, rate limiting, role-based access, and audit logs for payment and order-state changes. It should store payment references and transaction status rather than raw card details. Webhook authenticity and transaction verification must follow Paystack's current documentation.

7.3 Worker and Customer Trust

Workers should be onboarded and trained on order handling, price entry, receipts, substitutions, and customer privacy. A worker should only see the address and contact details needed for an assigned order. The service must define procedures for unavailable goods, price differences, late delivery, damaged items, worker non-response, cancellations, and disputes.

VIII. TRACEABILITY AND OPERATIONAL WORKFLOW

8.1 Market-Based Worker Network

Workers are associated with a participating market or market zone. Once an order is paid, the assignment service selects an eligible worker who is available at the chosen market. If the worker declines or does not respond within a configured period, the system should reassign the task or alert operations instead of leaving the order stalled.

8.2 Real-Time Shopping View

For the MVP, “real time” means that customers see item-level status updates as workers record them, with timestamps and optional purchase evidence. This can be implemented with periodic API refreshes or a managed real-time channel. Continuous GPS tracking is not necessary to validate the core service. The interface must distinguish an item marked “found” from an item confirmed as “purchased.”

8.3 Order Events and Audit Trail

Every material event should include the order, actor, timestamp, event type, and relevant details. Examples include payment verified, worker assigned, item unavailable, substitution requested, customer approval received, actual price recorded, delivery started, and delivery confirmed. This event history helps operations resolve disputes and allows the team to measure fulfilment performance.

IX. EVALUATION PLAN

The proposed system has not yet been empirically evaluated. The measures below are intended for a future pilot and must not be presented as achieved results before data is collected.

Evaluation area	Measure	Pilot method
Ingredient-list quality	Lists accepted with minor edits; missing or duplicate ingredients.	Compare AI suggestions with reviewed reference recipes and user feedback.
Price accuracy	Difference between estimate and actual purchase total.	Compare timestamped estimates with worker-confirmed receipts.
Fulfilment	Acceptance, shopping, delivery time and completion rate.	Log order-state timestamps during a controlled pilot.
Transparency	Unapproved substitutions and price-related complaints.	Audit approval events and post-order reports.
Payments	Verification success, duplicates, refund reconciliation time.	Test mode, then controlled low-value live tests.
Unit economics	Contribution per completed order after operating costs.	Measure worker, transport, processing, support and refund costs.

X. USER INTERFACE AND USER EXPERIENCE

10.1 Customer Experience Principles

The interface should keep the main task visible and reduce uncertainty. The home screen asks what the customer wants to cook. The recipe review screen shows serving count and ingredient quantities. A pantry step lets the user mark items as available and enter amounts. The market screen shows participating options and how recent the price data is. Checkout summarizes ingredient estimate, delivery fee, service fee, and the policy for price changes.

10.2 Worker Experience

The worker interface should be optimized for use while shopping in a busy market. Each item should display its requested quantity and unit, a simple status control, an actual-price field, and an action to report unavailability or request a substitute. The worker should be able to see the remaining list without navigating through unrelated information.

10.3 Customer Order Timeline

After checkout, the customer sees a clear sequence: payment verified, worker assigned, shopping started, item updates, approval required if applicable, shopping completed, out for delivery, and delivered. Notifications should be meaningful rather than excessive. If a customer must approve a substitute, the application should show the original item, proposed replacement, price difference, and a clear decision deadline.

FIGURE 2: Proposed Order Timeline

01 — ORDER CONFIRMED	02 — WORKER SHOPPING	03 — APPROVAL IF NEEDED	04 — OUT FOR DELIVERY
Payment verified; worker assigned.	Each item is updated with actual availability and price.	Customer reviews substitution or material price change.	Final receipt is available; delivery handover is tracked.

10.4 Data and Price Presentation

Price information should be legible and contextual. An estimated price should be labelled as such, with its market and observation date where available. Confirmed purchase prices should be visually distinct from estimates. The final receipt should itemize quantities, unit prices, line totals, delivery and service fees, approved substitutions, and any refund or adjustment.

10.5 Accessibility and Low-Connectivity Considerations

Use readable typography, high-contrast text, clear status labels, large touch targets, and concise prompts. The worker app should preserve the current shopping list when connectivity drops and synchronize updates when service returns. The system should clearly indicate when an update is pending so that users do not mistake a locally saved action for a server-confirmed purchase.

XI. IMPLEMENTATION AND TESTING

11.1 Implementation Roadmap

Phase	Deliverables	Exit condition
1. Discovery	Interview customers/workers; choose pilot market; test manual fulfilment.	Evidence that the service is useful and operationally feasible.
2. Core prototype	Meal form, serving selector, pantry subtraction, market choice, estimate.	Users can build and review a basket end to end.
3. Worker tools	Task list, item statuses, actual-price capture, substitution approval.	Test order can be fulfilled with an auditable receipt.
4. Payments	Paystack test mode, verification, webhooks, refund workflow.	Repeated test cases reconcile without duplicate fulfilment.
5. Controlled pilot	Limited market coverage, vetted workers, support and delivery.	Predefined reliability and trust thresholds are met.

XII. RESULTS AND DISCUSSION

12.1 Expected System Behaviour

The proposed system is expected to reduce the number of separate steps a customer performs to source ingredients, but that expectation must be tested. A successful end-to-end test should demonstrate that a dish can be translated into an editable list, quantities can be scaled and reduced by pantry stock, the basket can be priced from available observations, and a worker can fulfil the approved list while recording actual purchases.

12.2 Functional Testing

Functional testing should cover common and edge cases: a dish with multiple recipe variants; an ingredient the user already has in full; partial pantry quantities; ambiguous units; an item unavailable at the selected market; a worker who does not accept the order; a customer who declines a substitution; a repeated payment webhook; a payment with the wrong amount or currency; and a cancelled order requiring reconciliation.

12.3 Price Estimation Evaluation

For each pilot order, preserve the pre-checkout estimate and compare it with the actual purchase cost, excluding separately displayed fees. Report absolute and percentage differences, and analyze them by ingredient, market, and age of price observation. The team should avoid summarizing price accuracy with a single average if large errors for particular ingredients are hidden by small errors elsewhere.

12.4 Usability Evaluation

Observe whether customers can complete the meal-to-basket flow without help, understand which ingredients are missing, recognize estimated versus confirmed prices, and approve or decline a substitution. Worker testing should assess whether item entry can be completed quickly and accurately under real market conditions. Feedback should inform the next iteration of the interface and operating procedures.

12.5 Discussion

The platform's value proposition is the coordination of a physical shopping task rather than recipe generation alone. AI may reduce friction when creating a list, but operational reliability depends on human fulfilment, accurate item mapping, fresh price observations, clear payment rules, and responsive support. A narrowly scoped pilot is likely to provide more useful evidence than launching across many markets before these components are validated.

12.6 Limitations

The proposed design is not yet validated by production data. Recipe outputs may omit ingredients or suggest unsuitable quantities, market observations may be stale, and prices can change between estimation and purchase. Workers may encounter stock shortages, delays, or vendor differences. The platform must therefore make recipes editable, display price freshness, seek approval for material changes, and keep a traceable record of actual purchases.

XIII. CONCLUSION

This paper proposes a Smart Market Delivery Platform that turns a consumer's meal choice into a managed market-to-home ingredient service. The customer selects a dish and serving count, subtracts what is already available at home, chooses a participating market, reviews an estimate, pays through an integrated checkout, and follows a market worker as the missing ingredients are purchased and delivered. The AI model is positioned as a helper for recipe interpretation and ingredient normalization, while dated market observations and worker-confirmed purchases underpin price transparency.

The recommended MVP is deliberately focused: a limited set of dishes and markets, an editable shopping list, a worker interface, item-level price capture, Paystack verification, and a clear order timeline. The next step is to validate the service through customer interviews, manual fulfilment, and a controlled pilot before claiming accuracy, scalability, or profitability.

XIV. RISKS AND FUTURE WORK

14.1 Operational and Commercial Risks

The service is more operationally intensive than a market-price website. It requires recruiting and coordinating workers, handling multiple vendors, resolving substitutions, arranging delivery, supporting customers, and reconciling payment differences. Potential revenue sources include a service fee per order, a delivery fee based on distance or zone, and future vendor partnerships. The pilot must measure worker compensation, transport costs, payment fees, support, failed orders, and refunds before unit economics can be estimated credibly.

14.2 Privacy, Compliance, and Safety

The platform should collect only the customer information needed to complete an order, limit staff access to assigned tasks, and publish understandable cancellation, refund, and substitution policies. Applicable Nigerian data-protection, consumer, tax/accounting, and payment-provider obligations should be reviewed with qualified advisers before launch. API credentials must be secured and all payment operations must follow the provider's current documentation.

14.3 Future Enhancements

After validating the core workflow, future versions may add more dishes, additional markets, smarter worker assignment, saved household pantry lists, dietary preferences, repeat orders, vendor partnerships, improved price forecasting from historical observations, and native mobile applications. These features should be prioritized according to pilot evidence rather than added before the basic purchase-and-delivery service works reliably.

REFERENCES

Paystack. (n.d.). *Paystack API documentation*. https://paystack.com/docs/

Django Software Foundation. (n.d.). *Django REST framework documentation*. https://www.django-rest-framework.org/

MySQL Global Development Group. (n.d.). MySQL *documentation*. https://www.MySQL .org/docs/

OpenAI. (n.d.). *API documentation*. https://platform.openai.com/docs/ (or documentation for the selected AI model provider).

Note: These are implementation references. Before formal academic submission, supplement them with peer-reviewed research on food delivery logistics, recipe recommendation, digital marketplaces, price estimation, and trust in on-demand services.

Author contact: Hajarah Ayomide Ibrahim — hajbae01@gmail.com